

PLANT DISEASE CLASSIFICATION

PROJECT REPORT

PLANT DISEASE CLASSIFICATION USING DEEP LEARNING

PROJECT REPORT

LAB 14 – Complex Computing Activity of Computer vision

Student Names

- Sabiha Pirzadah
- Hafsa Naz
- Saira Jabeen
- Kainat Moin

Roll Numbers

- 23-AI-84
- 23-AI-56
- 23-AI-10
- 23-AI-48

Instructor

Engr. Hamza Farooqui

Institution

Dawood University of Engineering and Technology

Submission Date

14-May-2026

CONTENTS

S. No	Topic	Page no
1.	Introduction	03
2.	System Architecture	03-04
3.	Concepts Used	04
4.	Dataset Description	05
5.	Data Preprocessing & Augmentation	06
6.	Image Filtering & Visualization	07
7.	Model Architectures	07-08
8.	Training Configuration & Strategy	09
9.	Model Comparison Results	10
10.	Evaluation Results	11-13
11.	Image Upload and Prediction	14
12.	Challenges and Limitations	16
13.	Fairness and Ethical Considerations	17
14.	Setup and Instructions	17-18
15.	Conclusion	18

1 INTRODUCTION

Plant diseases are a major cause of agricultural loss worldwide and especially in countries like Pakistan where farming is a primary livelihood. Early and accurate detection of plant diseases can help farmers take timely action, reduce crop damage, and improve food security.

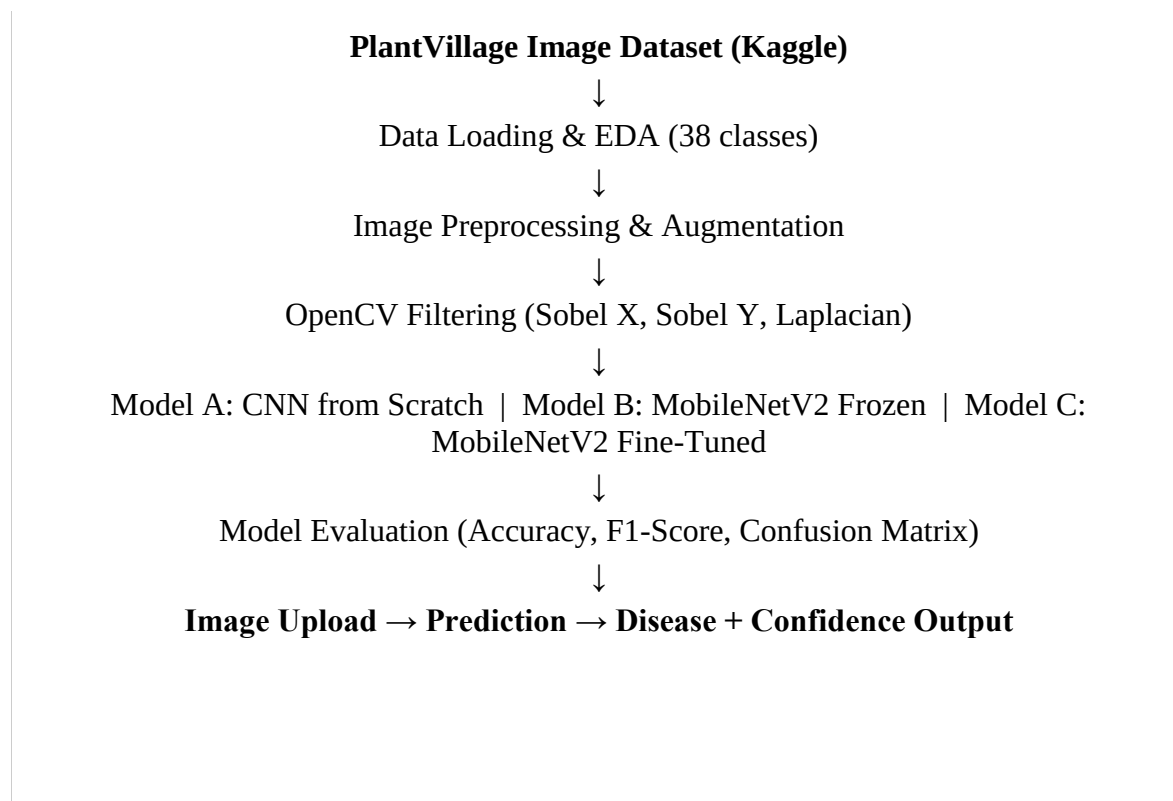
This project aims to build an automated plant disease classification system that can identify 38 different plant disease categories from leaf images. Three deep learning models are trained and compared: a custom CNN built from scratch, a frozen MobileNetV2 transfer learning model, and a fine-tuned MobileNetV2 model. The best performing model is then deployed for real-time leaf image prediction.

The project integrates image preprocessing using OpenCV filters, data augmentation, transfer learning, model evaluation, and a practical prediction interface — all within Google Colab using GPU acceleration.

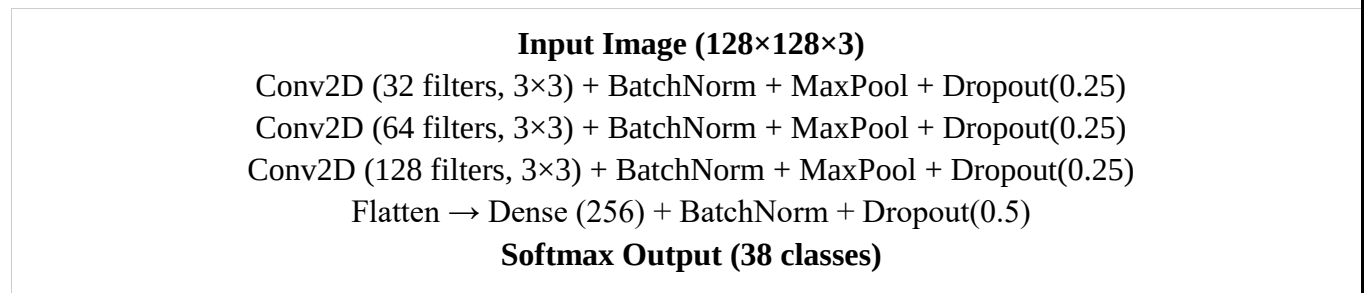
2 SYSTEM ARCHITECTURE

2.1 Pipeline

The complete pipeline of the project consists of the following stages:



2.2 Architectural Diagram — CNN from Scratch



3 CONCEPTS USED

The table below highlights what concepts have been used while creating this project:

S. No	Concept Used	How It Was Used
1.	Image Preprocessing	Resizing to 128×128, pixel normalization (/255), BGR to RGB conversion using OpenCV
2.	Edge Detection Filters	Sobel X, Sobel Y, and Laplacian filters applied via OpenCV to visualize leaf disease patterns
3.	Data Augmentation	Random flip (horizontal + vertical), rotation (0.2), zoom (0.2), contrast (0.2) using Keras layers
4.	Custom CNN (from scratch)	3-block Conv2D + BatchNorm + MaxPool + Dropout architecture, trained end-to-end
5.	Transfer Learning (Frozen)	MobileNetV2 pretrained on ImageNet, base frozen, only classification head trained
6.	Transfer Learning (Fine-Tuned)	Last 30 layers of MobileNetV2 unfrozen, trained with Adam (lr=1e-5)
7.	Feature Map Visualization	Conv1 feature maps extracted and visualized for healthy vs diseased leaves
8.	Model Evaluation	Accuracy, weighted F1-score, confusion matrix, per-class accuracy bar chart
9.	Classification Report	Per-class precision, recall, F1-score using sklearn; top 5 best & worst classes identified
10.	Model Persistence	Models saved as .keras files to Google Drive; best model loaded for evaluation
11.	Practical Deployment	Image upload via Google Colab; prediction with confidence score, annotated output image saved

4 DATASET DESCRIPTION

4.1 Dataset Source

The dataset used is the New Plant Diseases Dataset sourced from Kaggle (vip00000l/new-plant-diseases-dataset). It contains augmented leaf images organized into labeled folders, with each folder representing one plant-disease combination. The dataset was downloaded programmatically using the Kaggle API inside Google Colab.

4.2 Dataset Split

- Training set: 70,295 images across 38 classes
- Validation set: 17,572 images across 38 classes
- Total dataset: 87,867 images

4.3 Plant and Disease Classes (38 Total)

The dataset covers 14 plant species with a total of 38 classes including both diseased and healthy categories:

ID	Class Name	ID	Class Name
00	Apple — Apple scab	01	Apple — Black rot
02	Apple — Cedar apple rust	03	Apple — Healthy
04	Blueberry — Healthy	05	Cherry — Powdery mildew
06	Cherry — Healthy	07	Corn — Cercospora leaf spot
08	Corn — Common rust	09	Corn — Northern Leaf Blight
10	Corn — Healthy	11	Grape — Black rot
12	Grape — Esca (Black Measles)	13	Grape — Leaf blight
14	Grape — Healthy	15	Orange — Haunglongbing
16	Peach — Bacterial spot	17	Peach — Healthy
18	Pepper — Bacterial spot	19	Pepper — Healthy
20	Potato — Early blight	21	Potato — Late blight
22	Potato — Healthy	23	Raspberry — Healthy
24	Soybean — Healthy	25	Squash — Powdery mildew
26	Strawberry — Leaf scorch	27	Strawberry — Healthy
28	Tomato — Bacterial spot	29	Tomato — Early blight
30	Tomato — Late blight	31	Tomato — Leaf Mold
32	Tomato — Septoria leaf spot	33	Tomato — Spider mites
34	Tomato — Target Spot	35	Tomato — Yellow Leaf Curl Virus
36	Tomato — Mosaic virus	37	Tomato — Healthy

4.4 Exploratory Data Analysis (EDA)

EDA was conducted to understand the dataset structure before model training:

- Total number of classes identified: 38
- Sample images from each class visualized using matplotlib
- Class distribution examined to identify any imbalance between classes
- Dataset folder structure inspected programmatically using `os.walk()`
- One sample image displayed per class to confirm image quality and labeling

5 DATA PREPROCESSING & AUGMENTATION

5.1 Image Preprocessing

The following preprocessing steps were applied to every image before feeding into models:

- Image resizing to 128×128 pixels using `tf.keras.utils.image_dataset_from_directory`
- Pixel normalization: rescaling by $1/255$ using `tf.keras.layers.Rescaling`
- Color space conversion from BGR to RGB using `cv2.cvtColor` for OpenCV-loaded images
- Custom `process_image()` function created using OpenCV for manual image loading and processing

5.2 Data Augmentation

Augmentation was applied only to the training set to reduce overfitting and improve model generalization. A Keras Sequential augmentation pipeline was used:

- Random Flip: horizontal and vertical (`RandomFlip`)
- Random Rotation: factor of 0.2 radians (`RandomRotation`)
- Random Zoom: factor of 0.2 (`RandomZoom`)
- Random Contrast: factor of 0.2 (`RandomContrast`)

The augmentation is applied on-the-fly during training using `tf.data` pipeline with `num_parallel_calls=AUTOTUNE` and `prefetch(AUTOTUNE)` for optimal GPU utilization. Validation data is only normalized, not augmented.

6 IMAGE FILTERING & VISUALIZATION

6.1 OpenCV Filter Application

To analyze how convolutional filters respond to plant disease patterns, three classical image filters were applied using OpenCV on a randomly selected set of 5 disease classes:

- Sobel X Filter: detects vertical edges using kernel $\begin{bmatrix} -1, 0, 1 \\ -2, 0, 2 \\ -1, 0, 1 \end{bmatrix}$
- Sobel Y Filter: detects horizontal edges using kernel $\begin{bmatrix} -1, -2, -1 \\ 0, 0, 0 \\ 1, 2, 1 \end{bmatrix}$
- Laplacian Filter: detects all edges using kernel $\begin{bmatrix} 0, 1, 0 \\ 1, -4, 1 \\ 0, 1, 0 \end{bmatrix}$

Images were first converted to grayscale before filter application. Results were displayed in a 5×4 grid (Original, Sobel X, Sobel Y, Laplacian) titled 'Sobel & Laplacian Filter Response — 5 Disease Classes'.

6.2 CNN Feature Map Visualization

After training, the Conv1 layer feature maps were extracted and visualized for both a healthy and a diseased leaf image. An intermediate feature extraction model was built from the trained CNN to output activations at conv1. Eight feature maps were displayed side-by-side for healthy vs diseased leaves, showing how the model learns to differentiate disease patterns from healthy tissue.

7 MODEL ARCHITECTURES

7.1 Model A — Custom CNN from Scratch

A 3-block convolutional neural network was built entirely from scratch without any pretrained weights. The architecture is as follows:

Layer (Type)	Output Shape	Parameters
conv1 (Conv2D, 32 filters, 3×3, ReLU)	(None, 128, 128, 32)	896
BatchNormalization	(None, 128, 128, 32)	128
MaxPooling2D	(None, 64, 64, 32)	0
Dropout (0.25)	(None, 64, 64, 32)	0
conv2 (Conv2D, 64 filters, 3×3, ReLU)	(None, 64, 64, 64)	18,496
BatchNormalization	(None, 64, 64, 64)	256
MaxPooling2D	(None, 32, 32, 64)	0
Dropout (0.25)	(None, 32, 32, 64)	0

conv3 (Conv2D, 128 filters, 3×3, ReLU)	(None, 32, 32, 128)	73,856
BatchNormalization	(None, 32, 32, 128)	512
MaxPooling2D	(None, 16, 16, 128)	0
Dropout (0.25)	(None, 16, 16, 128)	0
Flatten	(None, 32768)	0
Dense (256, ReLU)	(None, 256)	8,388,864
BatchNormalization	(None, 256)	1,024
Dropout (0.5)	(None, 256)	0
Dense (38, Softmax)	(None, 38)	9,766
Total Parameters	8,493,798	32.40 MB

7.2 Model B — MobileNetV2 Frozen (Transfer Learning)

MobileNetV2 pretrained on ImageNet was used as a feature extractor. The entire base model was frozen (trainable=False) and only a custom classification head was trained on top.

- Base: MobileNetV2 (input 128×128×3, pretrained on ImageNet, include_top=False)
- Global Average Pooling 2D
- Dense layer (224 neurons, ReLU)
- BatchNormalization + Dropout (0.4)
- Softmax output (38 classes)
- Trainable parameters: 295,942 | Total: 2,554,374 (9.74 MB)

7.3 Model C — MobileNetV2 Fine-Tuned

The frozen MobileNetV2 model was extended by unfreezing the last 30 layers of the base model for domain adaptation to plant disease images.

- Base model: MobileNetV2 (last 30 of 154 layers unfrozen, first 124 frozen)
- Trainable layers after unfreezing: 30 layers of MobileNetV2 + full classification head
- Trained with lower learning rate (Adam 1e-5) to avoid destroying pretrained weights
- Trainable parameters: approximately 1.1 million | Total: 2,554,374

8 TRAINING CONFIGURATION & STRATEGY

8.1 Hyperparameters

Parameter	CNN Scratch	MobileNetV2 Frozen	MobileNetV2 Fine-Tuned
Optimizer	Adam	Adam	Adam
Learning Rate	1e-3	1e-3	1e-5
Batch Size	64	64	64
Image Size	128×128	128×128	128×128
Epochs	5 (max)	5 (max)	5 (max)
Loss Function	Sparse Categorical Crossentropy	Sparse Categorical Crossentropy	Sparse Categorical Crossentropy

8.2 Training Strategy

Two Keras callbacks were used during training to prevent overfitting and optimize convergence:

- EarlyStopping: monitors val_accuracy (CNN) or val_loss (MobileNetV2), patience=5, restores best weights automatically
- ReduceLROnPlateau: monitors val_loss, reduces learning rate by factor 0.5 when plateau detected, patience=3, minimum lr=1e-6 (CNN) / 1e-7 (Fine-Tuned)

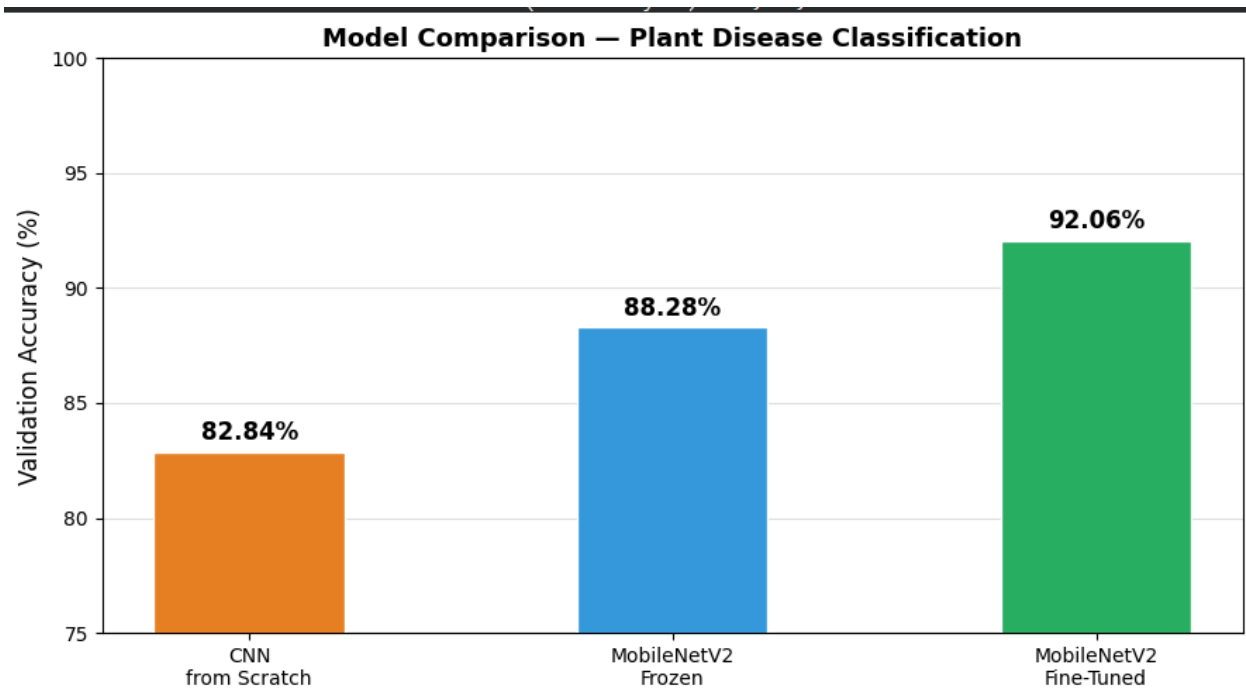
The training data pipeline uses tf.data with parallel preprocessing (AUTOTUNE) and prefetching for optimal GPU utilization. TensorFlow version 2.20.0 was used, with GPU acceleration confirmed (PhysicalDevice GPU:0 available in Colab).

9 MODEL COMPARISON RESULTS

After training all three models for 5 epochs each, the following results were obtained on the validation set (17,572 images):

Model	Architecture	Val Accuracy	Parameters	Time (min)	Epochs
CNN from Scratch	3-layer Custom CNN	82.84%	8,493,798	26.1	5
MobileNetV2 Frozen	MobileNetV2 + head	88.28%	2,554,374	28.3	5
MobileNetV2 Fine-Tuned	MobileNetV2 + head	92.06%	2,554,374	26.1	5

The MobileNetV2 Fine-Tuned model achieved the best validation accuracy of 92.06% and was selected as the best model, saved to Google Drive at /content/drive/MyDrive/PlantDisease/best_model.keras for evaluation and prediction.



10 EVALUATION RESULTS

10.1 Overall Accuracy & F1-Score

Inference was run on the full validation set (17,572 images) using the best saved model (MobileNetV2 Fine-Tuned):

- Overall Accuracy: 92.06%
- Weighted F1-Score: 0.9206

10.2 Training Curves

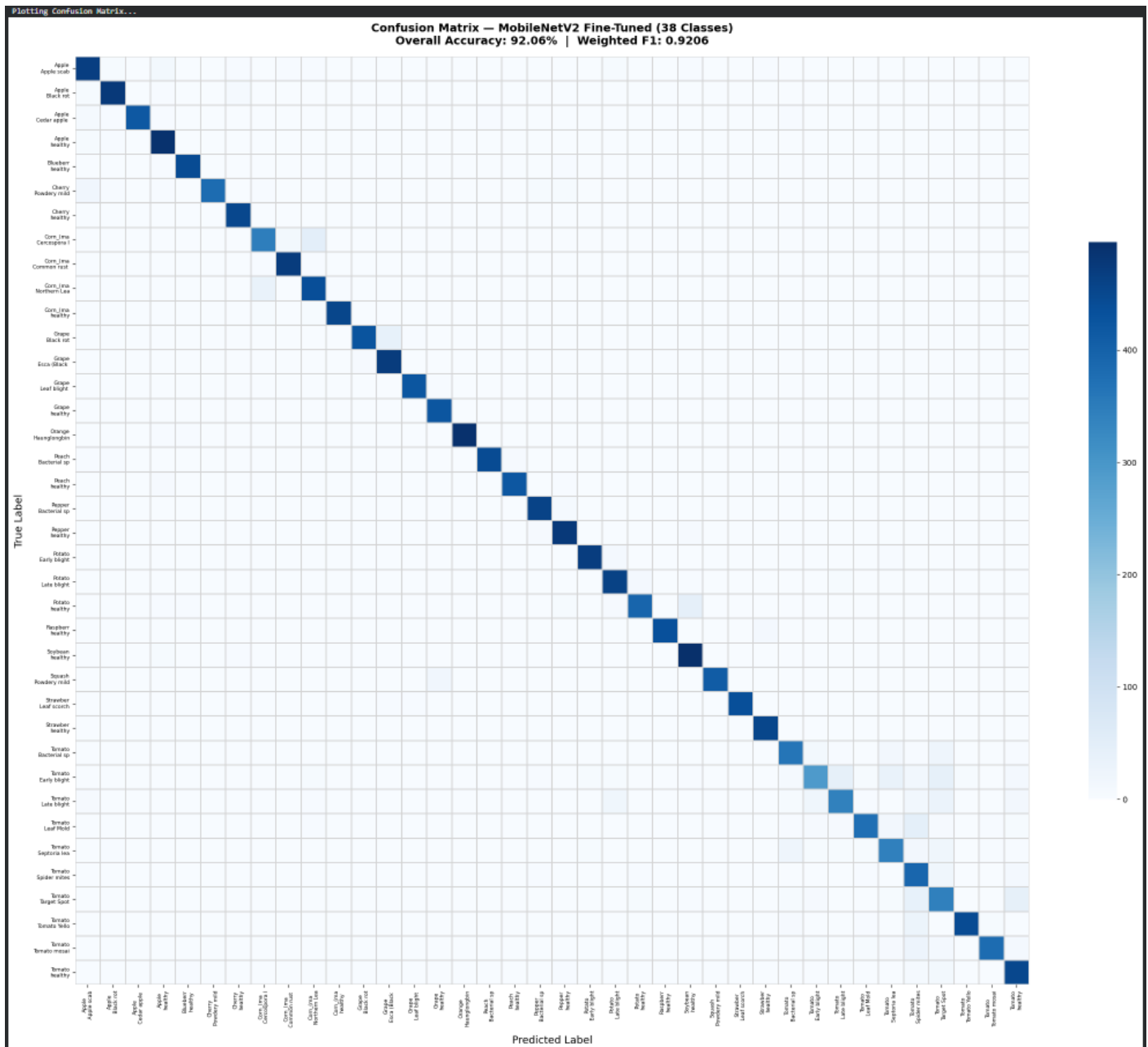
Training curves (accuracy and loss) were plotted for all three models using matplotlib. The MobileNetV2 Fine-Tuned model showed consistent epoch-over-epoch improvement with both training and validation metrics converging together, indicating good generalization with limited overfitting.

CNN from Scratch showed higher training accuracy but more variable validation accuracy (82.84% best on epoch 4), which indicates some overfitting despite regularization. MobileNetV2 Frozen was more stable, achieving 88.28% consistently from the first epoch onwards.

10.3 Confusion Matrix

A 38×38 confusion matrix was generated using seaborn heatmap and saved to Google Drive. Key observations:

- Strong diagonal (high per-class accuracy) across most classes, especially non-Tomato categories
- Most confusion occurs within Tomato disease classes which share similar visual features
- Classes like Strawberry Leaf scorch and Orange Haunglongbing showed near-perfect classification



10.4 Per-Class Accuracy

A color-coded bar chart was generated showing per-class accuracy for all 38 classes:

- Green bars ($\geq 85\%$): majority of classes including all fruit and grain diseases
- Red bars ($< 85\%$): primarily Tomato subclasses due to visual similarity
- Overall accuracy line (92.06%) overlaid as a dashed reference

10.5 Top 5 Best & Worst Performing Classes

Class	Precision	Recall	F1-Score
TOP 5 BEST PERFORMING			
Strawberry — Leaf scorch	0.998	0.984	0.991
Orange — Haunglongbing (Citrus greening)	0.996	0.980	0.988
Corn (maize) — Healthy	0.993	0.978	0.986
Grape — Leaf blight (Isariopsis Leaf Spot)	0.984	0.986	0.985
Raspberry — Healthy	0.995	0.975	0.985
TOP 5 WORST PERFORMING			
Tomato — Early blight	0.827	0.608	0.701
Tomato — Target Spot	0.667	0.751	0.706
Tomato — Spider mites (Two-spotted)	0.659	0.903	0.762
Tomato — Late blight	0.840	0.739	0.786
Tomato — Septoria leaf spot	0.815	0.789	0.802

11 IMAGE UPLOAD AND PREDICTION

The system provides a complete prediction interface implemented in Google Colab:

- User uploads a leaf image using `files.upload()` from `google.colab`
- Image is read using `OpenCV (cv2)`
- Image is resized to 128×128
- Image is converted from BGR to RGB format
- Pixel values are normalized (/255)
- The processed image is passed to the trained `MobileNetV2 Fine-Tuned` model
- The model predicts:
 - Plant Name
 - Disease Name
 - Confidence Score (%)

The prediction result is displayed directly on the uploaded image using `cv2.putText()` with colored annotations:

- Green overlay for confidence greater than 70%
- Orange overlay for lower confidence predictions

The annotated image is displayed in the notebook and saved to Google Drive for future reference.

11.1 Unseen Validation Image Testing

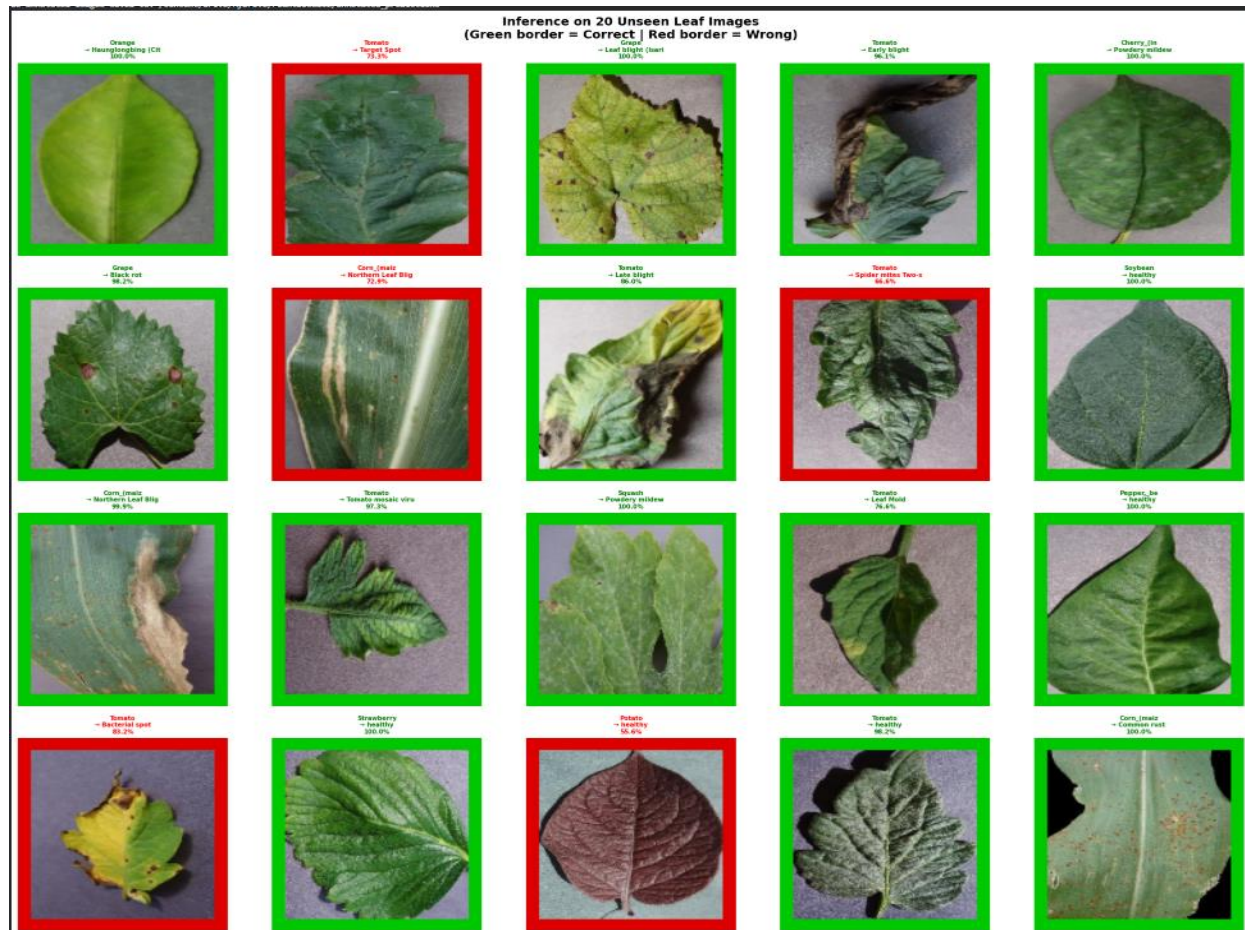
To evaluate real-world generalization performance, the trained model was tested on **20 randomly selected unseen validation images** taken from different plant disease classes that were not manually inspected during training.

Results:

- Correct Predictions: **15**
- Incorrect Predictions: **5**
- Approximate Accuracy on Unseen Samples: **75%**

Most incorrect predictions occurred between visually similar disease categories, especially among Tomato leaf diseases where symptoms such as spots, blight patterns, and discoloration overlap significantly.

This experiment demonstrated that the model generalized reasonably well to unseen validation samples while still facing challenges with highly similar disease classes.



11.2 Real Image Upload Testing

In addition to validation testing, manually selected plant leaf images from a personal folder were uploaded separately to test the practical prediction capability of the system.

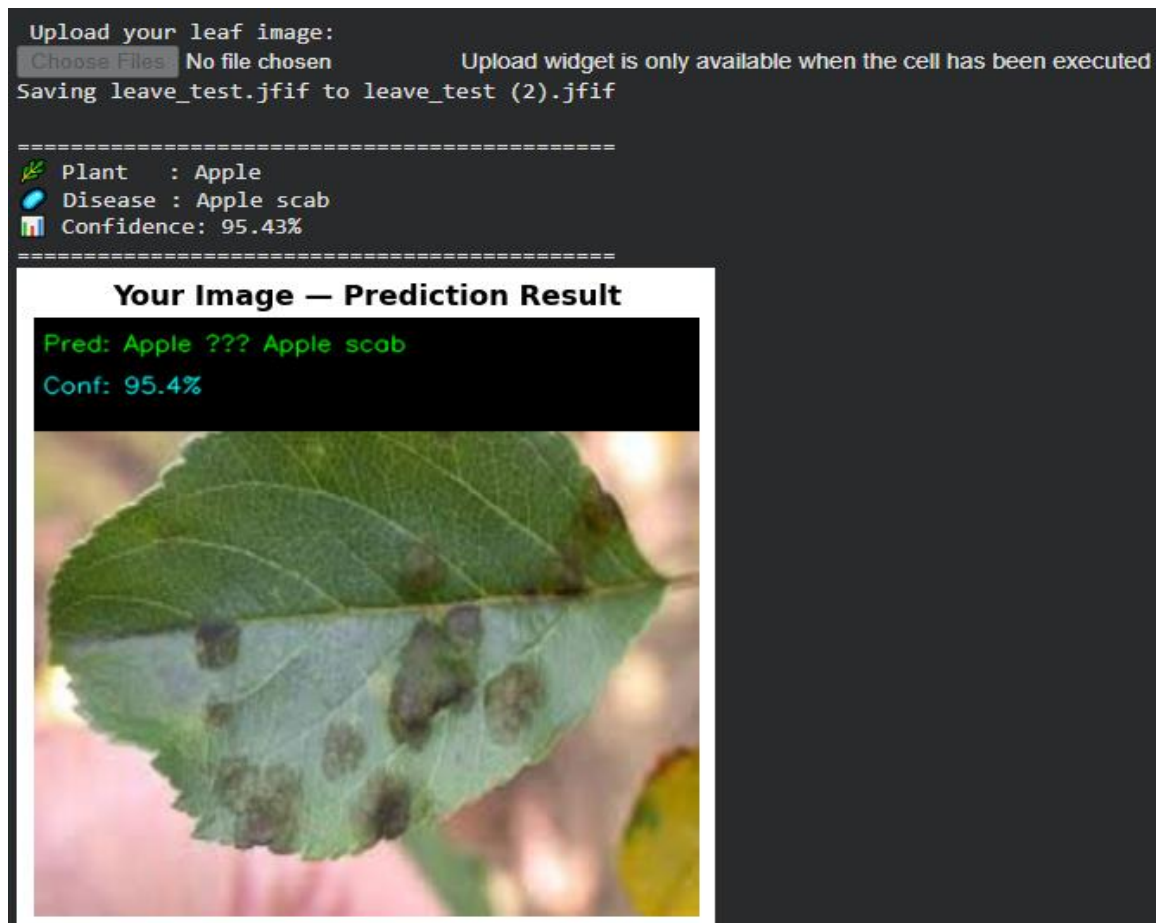
The uploaded images were correctly classified by the model with high confidence scores, demonstrating that the trained MobileNetV2 Fine-Tuned model can successfully perform real-image disease prediction outside the standard validation pipeline.

Example:

An uploaded Apple leaf image was correctly predicted as:

Apple → Apple Scab
Confidence Score: 95.43%

The prediction interface successfully handled real user-uploaded images and produced visually annotated outputs inside the Google Colab notebook environment.



12 CHALLENGES AND LIMITATIONS

12.1 Challenges

- Tomato disease classes are visually very similar (e.g. Early blight vs Late blight vs Target Spot), causing the most classification confusion
- CNN from Scratch showed inconsistent validation accuracy across epochs (e.g. epoch 2: 40.27%, epoch 4: 82.84%) due to learning rate not being reduced early enough
- High dimensional input ($128 \times 128 \times 3 \times 70,295$ training images) required GPU acceleration for feasible training times (~26 min per model)
- MobileNetV2 Frozen epoch 1 val accuracy (86.30%) was already very high, leaving limited room for improvement in subsequent epochs

12.2 Limitations

- Model not optimized for real-time mobile or edge deployment; requires GPU for fast inference
- Dataset is augmented, meaning it was created from a limited set of real images; performance on true real-world unseen plant images may differ
- Only 5 training epochs used due to time and compute constraints; more epochs could improve CNN from Scratch performance

- Image size reduced to 128×128 (from the standard 224×224) to speed up training; this may lose fine-grained disease texture details
- 38 classes with only 5 epochs may not be sufficient for CNN from Scratch to fully converge

13 FAIRNESS AND ETHICAL CONSIDERATIONS

- The dataset may favor certain plant species (Tomato has 10 classes) which could bias the model toward over-representing Tomato-related predictions
- Incorrect disease classification could lead farmers to apply wrong pesticide treatments, causing crop loss or environmental harm
- Model should display confidence scores always, so that users can make informed decisions rather than blindly trusting the output
- The system is intended to assist farmers and agricultural experts, not replace professional agronomic judgment
- Deployment in regions with different plant varieties (e.g. Pakistan vs USA) should be validated with locally collected data

14 SETUP AND RUN INSTRUCTIONS

14.1 Environment Setup

Google Colab is the recommended platform. Kaggle API credentials must be configured as Colab secrets (KAGGLE_USERNAME and KAGGLE_KEY).

Required Libraries (all pre-installed in Colab):

- TensorFlow 2.20.0
- scikit-learn (confusion_matrix, classification_report, accuracy_score, f1_score)
- OpenCV (cv2) — image loading, resizing, filtering, annotation
- Matplotlib, Seaborn — visualization and plotting
- NumPy, Pandas — array operations and tabular results
- kaggle — dataset download via API

14.2 Hardware Requirements

- GPU Required (Colab T4 GPU is sufficient)
- Minimum 12 GB RAM (Colab standard runtime)
- Google Drive for model and output storage

14.3 Running the Project

1. Mount Google Drive using `drive.mount('/content/drive')`
2. Set Kaggle API credentials in Colab Secrets
3. Run dataset download and unzip cells
4. Run EDA, preprocessing and augmentation cells
5. Run CNN from Scratch training cell (Model A)
6. Run MobileNetV2 Frozen training cell (Model B)
7. Run MobileNetV2 Fine-Tuned training cell (Model C)
8. Run model comparison and evaluation cells
9. Run image upload cell and provide a leaf image for prediction

15 CONCLUSION

This project demonstrates how deep learning can be effectively applied to the real-world problem of plant disease classification. Three models were implemented and compared: a custom CNN built from scratch achieved 82.84% validation accuracy, MobileNetV2 with frozen weights achieved 88.28%, and MobileNetV2 with fine-tuning of the last 30 layers achieved the best result of 92.06% accuracy with a weighted F1-score of 0.9206.

The fine-tuned transfer learning approach proved most effective because it combines the power of ImageNet-pretrained feature representations with domain-specific adaptation to plant leaf images. The project also demonstrates the value of combining multiple evaluation techniques: confusion matrices, per-class accuracy charts, top-5 best and worst class analysis, and real-image inference with confidence scores.

OpenCV-based filtering (Sobel X, Sobel Y, Laplacian) and feature map visualization added interpretability to the pipeline by showing how the model responds to edge patterns in diseased vs healthy leaves. The practical prediction interface allows any user to upload a leaf photograph and receive an instant, confidence-scored disease diagnosis.

Future improvements could include training for more epochs, using higher resolution inputs (224×224), experimenting with EfficientNet or ResNet architectures, adding Grad-CAM visualizations for explainability, and collecting real-world leaf images from Pakistan for local validation.



The End